



COMP – 304

Summer 2026



Mastering Navigation in Android Apps with Jetpack Navigation

Objectives:

- ☐ Discuss the principles and components of Jetpack Navigation.
- ☐ Implement navigation between different Compose destinations.
- ☐ Pass arguments between destinations within an Android app.
- ☐ Design responsive navigation for foldables and large screens.
- ☐ Utilize the resizable emulator to test navigation on various screen sizes.



Introduction to Navigation in Android Apps

- ❑ Navigation is essential in Android apps for moving between different screens.
- ❑ Modern applications often contain multiple screens to manage various functionalities.
- ❑ Introduction to Jetpack Compose for navigation, focusing on its use in modern Android development.
- ❑ Transitioning from apps with single screens to multi-screen setups increases user engagement.



Navigation Overview with Jetpack Compose

- ❑ Jetpack Navigation simplifies the implementation of complex navigation patterns in Android apps.
- ❑ Supports both traditional XML-based UIs and modern Jetpack Compose UIs.
- ❑ We will focus on using the Jetpack Compose Navigation library.
- ❑ **NavController** manages navigation logic, while **NavHost** is the container for destinations.
- ❑ Example application context: Navigating within a simple Pets app.



Setting Up Jetpack Navigation

1. Step 1: **Add Jetpack Navigation Compose dependency** to your `libs.versions.toml` and `build.gradle.kts`.
2. Step 2: Sync your project with Gradle to download the necessary libraries.
3. Step 3: **Create instances of NavController and NavHost** to manage navigation flows.
4. Step 4: NavController is crucial for managing stack of screens (composables).
5. Step 5: NavHost acts as a container for all composables that serve as destinations.



Code Example - Adding Navigation Dependencies

- ❑ Add the following line to your `libs.versions.toml`:

```
compose-navigation =  
    "androidx.navigation:navigation-compose:2.7.2"
```
- ❑ Update your **build.gradle.kts** file:

```
implementation(libs.compose.navigation)
```
- ❑ This setup allows the use of navigation components in the project.



Defining Navigation Routes Using Sealed Classes

- ❑ Sealed classes provide a **safe way to define fixed types of routes** in your app.
- ❑ **Define each screen as an object** within the sealed class to ensure type safety.
- ❑ Example: **Screens.kt** with routes for `PetsScreen` and `PetDetailsScreen`.
- ❑ **Sealed classes help prevent errors** during navigation by limiting the routes to predefined options.
- ❑ **Each object in the sealed class represents a unique destination** in your navigation graph.



Code Example - Defining Routes in a Sealed Class

❑ File: **Screens.kt**

```
sealed class Screens(val route: String) {  
    object PetsScreen : Screens("pets")  
    object PetDetailsScreen : Screens("petDetails")  
}
```

- ❑ This code snippet **defines two routes**: one for the **PetsScreen** and one for the **PetDetailsScreen**.



Implementing NavController and NavHost in Compose

- ❑ **rememberNavController()** is used to create and remember an instance of NavController across composables.
- ❑ **NavHost** is defined within your main layout to specify the navigation structure.
- ❑ **startDestination** in NavHost defines the first screen to display.
- ❑ **NavController** uses the routes defined in the sealed class to navigate between composables.
- ❑ This setup centralizes navigation logic in your main UI composable.



Code Example - Setup NavController and NavHost

❑ Implementation in Compose:

@Composable

```
fun AppNavigation() {  
    val navController = rememberNavController()  
    NavHost(navController = navController, startDestination =  
Screens.PetsScreen.route) {  
        composable(Screens.PetsScreen.route) { PetsScreen() }  
        composable(Screens.PetDetailsScreen.route) {  
PetDetailsScreen() }  
    }  
}
```

❑ This code creates a navigation graph with two destinations: **PetsScreen** and **PetDetailsScreen**.



Building the PetsScreen Composable

- ❑ Design PetsScreen with a Scaffold to provide structure for app bars and content.
- ❑ Use TopAppBar for consistent top navigation elements across screens.
- ❑ Implement PetList inside the Scaffold's content to display a list of pets.
- ❑ Add navigation to PetDetailsScreen via callbacks triggered by user interactions.
- ❑ Provide a complete user interface code for PetsScreen incorporating navigation.



Code Example - PetsScreen Composable

- ❑ Example of PetsScreen implementation in Jetpack Compose:

@Composable

```
fun PetsScreen(navController: NavController) {  
    Scaffold(  
        topBar = { TopAppBar(title = { Text("Pets") }) },  
        content = { PetList(navController) }  
    )  
}
```

- ❑ This composable sets up the top bar and the content area where the list of pets will be displayed.



Navigating to Compose Destinations

- ❑ Understanding navigation from one screen to another using **NavController**.
- ❑ Use **NavController.navigate()** to move to a specific destination.
- ❑ Handling user interactions to trigger navigation between screens.
- ❑ Example: Clicking on a pet in the PetsScreen navigates to the PetDetailsScreen.
- ❑ Passing unique identifiers or objects to destinations to maintain state and display relevant data.



Code Example - Triggering Navigation

- ❑ Triggering navigation to PetDetailsScreen when a pet item is clicked:

@Composable

```
fun PetList(navController: NavController, pets: List<Cat>) {  
    LazyColumn {  
        items(pets) { pet ->  
            PetListItem(pet, onClick = {  
  
                navController.navigate(Screens.PetDetailsScreen.route)  
            })  
        }  
    }  
}
```

- ❑ This code uses a **LazyColumn** to list pets and set up click actions to navigate.



Defining the PetDetailsScreen Composable

- ❑ Layout and structure of PetDetailsScreen for displaying detailed pet information.
- ❑ Using **AsyncImage** for loading images asynchronously within the screen.
- ❑ Incorporating back navigation using **IconButton** with a back arrow.
- ❑ Employing **Scaffold** to provide a consistent layout that includes a top app bar.
- ❑ Explanation of compositional setup for PetDetailsScreen, integrating image and text components.



Code Example - PetDetailsScreen Composable

- ❑ Implementation of PetDetailsScreen with navigation controls:

@Composable

```
fun PetDetailsScreen(navController: NavController, petId: String) {  
    Scaffold(  
        topBar = {  
            TopAppBar(  
                title = { Text("Pet Details") },  
                navigationIcon = {  
                    IconButton(onClick = { navController.popBackStack() }) {  
                        Icon(imageVector = Icons.Filled.ArrowBack, contentDescription = "Back")  
                    }  
                }  
            )  
        },  
        content = { PetDetailsContent(petId) }  
    )  
}
```

- ❑ This composable includes navigation logic to return to the previous screen.



Passing Arguments to Destinations

- ❑ Techniques for passing data between destinations in Jetpack Compose Navigation.
 - **Utilizing route arguments** to pass data such as IDs or full objects.
 - **Decoding passed arguments** in destination screens to retrieve and display data.
- ❑ Example: Passing a pet's ID from PetsScreen to PetDetailsScreen.
- ❑ Ensuring data consistency and type safety when passing and retrieving arguments.



Code Example - Passing Arguments

- ❑ Passing and retrieving a pet ID as a navigation argument:

```
// In PetList, triggering navigation with an argument
```

```
navController.navigate("${Screens.PetDetailsScreen.route}/$petId")
```

```
// In NavHost, receiving the argument
```

```
composable(
```

```
    "${Screens.PetDetailsScreen.route}/{petId}",
```

```
    arguments = listOf(navArgument("petId") { type =  
        NavType.StringType })
```

```
) { backStackEntry ->
```

```
    PetDetailsScreen(navController,  
        backStackEntry.arguments?.getString("petId")!!)
```

```
}
```

- ❑ This setup demonstrates handling navigation with parameters to pass and retrieve data.



Handling Navigation in Foldables and Large Screens

- ❑ Challenges and strategies for managing navigation on large screens and foldable devices.
- ❑ Using responsive layouts to adapt to different screen sizes and orientations.
- ❑ Implementing **NavigationRail**, **NavigationDrawer**, and **BottomNavigation** based on screen size.
- ❑ Example: Adjusting navigation components dynamically based on the device's screen state.
- ❑ Ensuring a seamless user experience across various device types.



Code Example - Responsive Navigation Setup

- ❑ Example setup for responsive navigation using **WindowSizeClass** and device type:

@Composable

```
fun ResponsiveNavigation(navController: NavController,  
    windowSizeClass: WindowSizeClass) {  
    when (windowSizeClass.widthSizeClass) {  
        Compact -> BottomNavigationBar(navController)  
        Medium, Expanded -> NavigationRail(navController)  
    }  
}
```

- ❑ This composable adjusts navigation elements based on the window size.

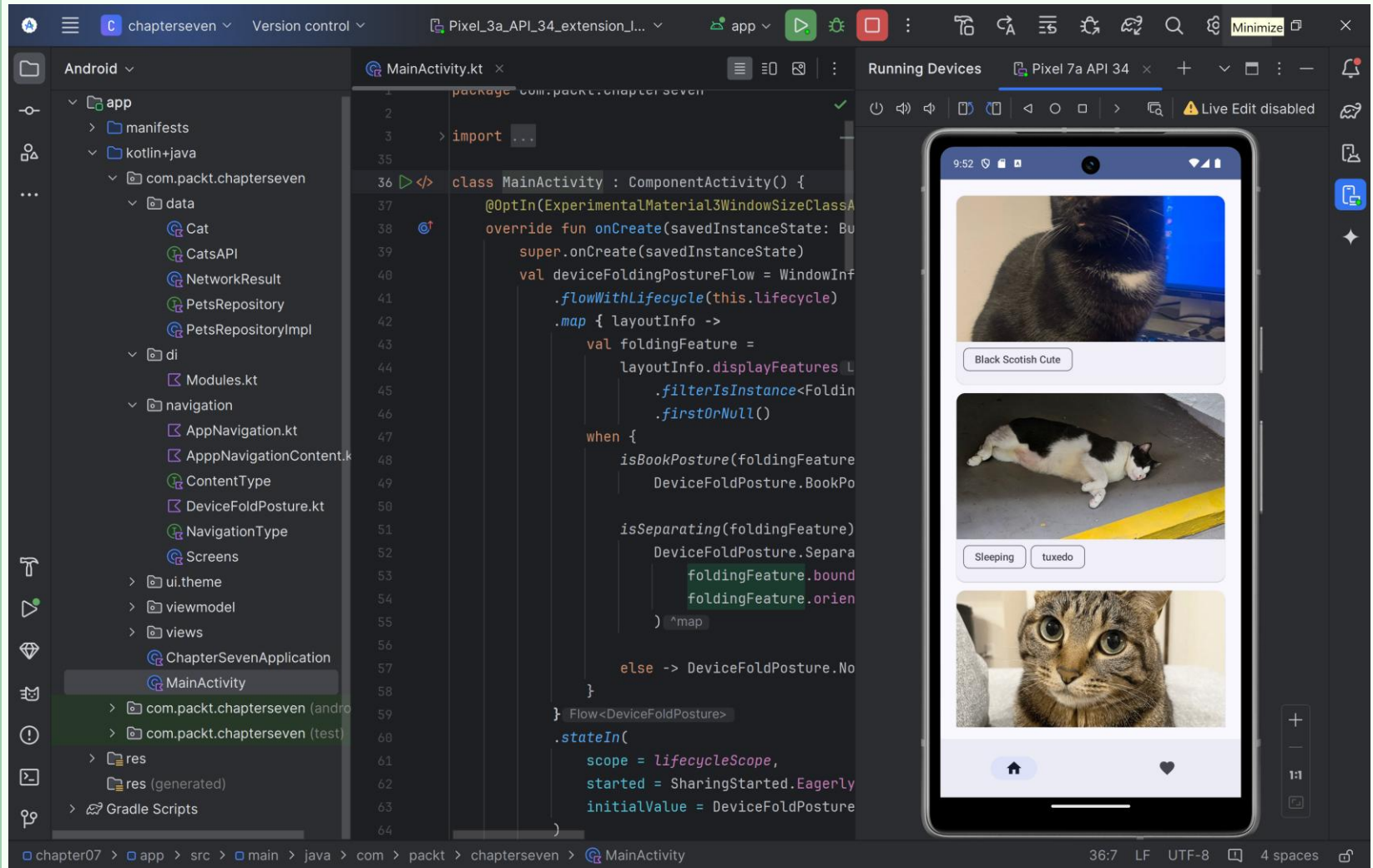


Summary and Next Steps in Navigation

- ❑ Recap of key concepts in Jetpack Compose Navigation:
 - Setup
 - route definition
 - argument passing
- ❑ Best practices for effective navigation in Android apps



chapter07 Example





References

- ❑ Textbook Chapter 7
- ❑ <https://developer.android.com/guide/navigation>